

FULL STACK DEVELOPER INTERVIEW PREPARATION KIT & ROADMAP

A Comprehensive 6-8 Month Mastery Guide with DSA & LeetCode Code Solutions

DSA Coding | Database Engineering | React 18+ | Java 21 Spring Boot | Docker & K8s | AWS | System Design

Target Stack	DSA Patterns, LeetCode Solutions, Database Engineering, React, Spring Boot, K8s, AWS, System Design
Total Roadmap Topics	265 Detailed Technical Topics & Coding Challenges
Estimated Hours	~600 - 750 Learning & Practice Hours
Target Duration	6 to 8 Months (15-20 hours/week)
Projects Included	4 Production-Grade Applications (EMS, E-Commerce, Real-Time Chat, Financial Banking)

Mastery Roadmap Architecture (10 Modules)

This preparation guide is organized into 10 core modules. Follow the recommended sequence for optimal retention and skill building.

DSA & Coding Problems	Master Big-O analysis, Two Pointers, Sliding Window, Monotonic Stacks, Trees, Graphs, Dynamic Programming, and Backtracking with LeetCode solutions.	60-80 hrs
Database Engineering & SQL/NoSQL	Master PostgreSQL/MySQL, B+Tree indexing, EXPLAIN ANALYZE query tuning, MVCC, Transactions, Isolation levels, HikariCP, MongoDB & Cassandra.	60-80 hrs
Java Core & Advanced	Master Java memory management, JVM internals, collections under the hood, multithreading, streams, and design patterns.	60-80 hrs
Spring Boot & Backend	Deep dive into Spring Core, IoC/DI, JPA/Hibernate, Spring Security, JWT, OAuth2, and Transaction Management.	60-80 hrs
React & Frontend	Build scalable SPAs with React 18, Custom Hooks, Redux Toolkit, Zustand, React Query, TypeScript, and RTL.	60-80 hrs
DevOps & Infrastructure	Containerize apps with multi-stage Docker builds, orchestrate with Kubernetes, Nginx, and build GitHub Actions CI/CD pipelines.	60-80 hrs
AWS Cloud Architecture	Deploy enterprise workloads across AWS EC2, IAM, VPC, S3, RDS, CloudWatch, ALB, and Auto Scaling.	60-80 hrs
Monitoring & APM	Observe distributed systems using Prometheus, Grafana, ELK Stack, Dynatrace, JMeter, and YourKit profiling.	60-80 hrs
System Design & Architecture	Master Low-Level Design (SOLID, GoF) and High-Level Design (Microservices, Kafka, Redis, Rate Limiting, CAP, Sharding).	60-80 hrs
Projects Showcase	4 Production-level projects with complete architecture, code structure, deployment pipelines, and interview walkthroughs.	60-80 hrs

Phase 1: Data Structures, Algorithms (DSA) & Coding Challenges

Topic 1: Complexity Analysis & Math Essentials — Big-O, Big-Omega, Big-Theta & Time/Space Trade-offs

Priority: High | Difficulty: Beginner | Est. Hours: 5h | Prerequisite: Basic Programming Syntax

Senior Interview Question: What is the amortized time complexity of dynamic array resizing? Contrast $O(1)$ vs $O(N)$ operations.

Real-World Usecase: Evaluating algorithmic efficiency before writing high-volume production loops.

■ **Coding Challenge: LeetCode #1 - Two Sum | Complexity: Time: $O(N)$ | Space: $O(N)$**

```
// Java Solution Code:
public int[] twoSum(int[] nums, int target) {
    Map map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int comp = target - nums[i];
        if (map.containsKey(comp)) return new int[]{map.get(comp), i};
        map.put(nums[i], i);
    }
    return new int[0];
}
```

Target Project: Employee Management System | **Resources:** Grokking Algorithms, LeetCode Explore Cards

Topic 2: Arrays & Strings Patterns — Two-Pointer Technique (Opposite End & Same Direction)

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Array Traversal

Senior Interview Question: How do you solve 3Sum in $O(N^2)$ time using two pointers without duplicate triplets?

Real-World Usecase: Eliminating nested $O(N^3)$ loops in array data transformation services.

■ **Coding Challenge:** LeetCode #15 - 3Sum | **Complexity:** Time: $O(N^2)$ | Space: $O(1)$

```
// Java Solution Code:
public List<int> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<int> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue;
        int l = i + 1, r = nums.length - 1;
        while (l < r) {
            int sum = nums[i] + nums[l] + nums[r];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                while (l < r && nums[l] == nums[l+1]) l++;
                while (l < r && nums[r] == nums[r-1]) r--;
                l++; r--;
            } else if (sum < 0) l++; else r--;
        }
    }
    return res;
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode 3Sum, Container With Most Water

Topic 3: Arrays & Strings Patterns — Sliding Window Pattern (Fixed & Dynamic Window Sizes)

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 7h | **Prerequisite:** Two Pointers

Senior Interview Question: How to find the Longest Substring Without Repeating Characters in $O(N)$ time?

Real-World Usecase: Real-time rate-limiting sliding windows and log stream pattern parsing.

■ **Coding Challenge:** LeetCode #3 - Longest Substring Without Repeating Characters | **Complexity:** Time: $O(N)$ | Space: $O(\min(N, M))$

```
// Java Solution Code:
public int lengthOfLongestSubstring(String s) {
    Map map = new HashMap<>();
    int maxLen = 0, l = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (map.containsKey(c)) l = Math.max(l, map.get(c) + 1);
        map.put(c, r);
        maxLen = Math.max(maxLen, r - l + 1);
    }
    return maxLen;
}
```

Target Project: Chat Application | **Resources:** LeetCode Sliding Window Patterns

Topic 4: Arrays & Strings Patterns — Prefix Sum Arrays & Difference Arrays

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 5h | **Prerequisite:** Arrays

Senior Interview Question: How to find the total number of continuous subarrays whose sum equals K in $O(N)$ time?

Real-World Usecase: Financial ledger sub-range totals and high-frequency analytical aggregations.

■ **Coding Challenge:** LeetCode #560 - Subarray Sum Equals K | **Complexity:** Time: $O(N)$ | Space: $O(N)$

```
// Java Solution Code:
public int subarraySum(int[] nums, int k) {
    Map map = new HashMap<>();
    map.put(0, 1);
    int sum = 0, count = 0;
    for (int n : nums) {
        sum += n;
        if (map.containsKey(sum - k)) count += map.get(sum - k);
        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}
```

Target Project: Banking System | **Resources:** LeetCode Subarray Sum Equals K

Topic 5: Linked List Data Structures — Singly & Doubly Linked Lists (In-place Reversal)

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Pointers & References

Senior Interview Question: How do you reverse a Singly Linked List iteratively in $O(1)$ auxiliary space?

Real-World Usecase: Building LRU Cache node linked structures and undo-redo history buffers.

■ **Coding Challenge:** LeetCode #206 - Reverse Linked List | **Complexity:** Time: $O(N)$ | Space: $O(1)$

```
// Java Solution Code:
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Reverse Linked List

Topic 6: Linked List Data Structures — Fast & Slow Pointers (Floyd's Cycle Detection Algorithm)

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 5h | **Prerequisite:** Linked List Basics

Senior Interview Question: How to detect a cycle in a Linked List and find the cycle starting node in $O(1)$ space?

Real-World Usecase: Detecting circular dependencies in workflow execution graphs.

■ **Coding Challenge:** LeetCode #142 - Linked List Cycle II | **Complexity:** Time: $O(N)$ | Space: $O(1)$

```
// Java Solution Code:
public ListNode detectCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            ListNode p = head;
            while (p != slow) { p = p.next; slow = slow.next; }
            return p;
        }
    }
    return null;
}
```

Target Project: Banking System | **Resources:** LeetCode Linked List Cycle II

Topic 7: Stack & Monotonic Stack — Stack Expressions & Monotonic Stack

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 7h | **Prerequisite:** Arrays & Stacks

Senior Interview Question: How does a Monotonic Stack solve 'Daily Temperatures' in $O(N)$ time?

Real-World Usecase: Evaluating algebraic expressions, compiler AST parsing, and stock price peak detection.

■ **Coding Challenge:** LeetCode #739 - Daily Temperatures | **Complexity:** Time: $O(N)$ | Space: $O(N)$

```
// Java Solution Code:
public int[] dailyTemperatures(int[] T) {
    Stack st = new Stack<>();
    int[] res = new int[T.length];
    for (int i = 0; i < T.length; i++) {
        while (!st.isEmpty() && T[i] > T[st.peek()]) {
            int idx = st.pop();
            res[idx] = i - idx;
        }
        st.push(i);
    }
    return res;
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Monotonic Stack Pattern

Topic 8: Queue & Monotonic Queue — Circular Queue, Deque & Monotonic Queue

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 6h | **Prerequisite:** Queue Basics

Senior Interview Question: How to solve Sliding Window Maximum using a Monotonic Deque in $O(N)$ time?

Real-World Usecase: Real-time rolling metric maximums in event stream processors.

■ **Coding Challenge:** LeetCode #239 - Sliding Window Maximum | **Complexity:** Time: $O(N)$ | Space: $O(K)$

```
// Java Solution Code:
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque dq = new ArrayDeque<>();
    int[] res = new int[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() == i - k) dq.pollFirst();
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}
```

Target Project: Chat Application | **Resources:** LeetCode Sliding Window Maximum

Topic 9: Binary Search & Search Space — Binary Search on Sorted Arrays & Rotated Arrays

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 7h | **Prerequisite:** Sorting Basics

Senior Interview Question: How to search a target in a Rotated Sorted Array in $O(\log N)$ time?

Real-World Usecase: Optimizing resource allocation thresholds using $\log(N)$ decision boundaries.

■ **Coding Challenge:** LeetCode #33 - Search in Rotated Sorted Array | **Complexity:** Time: $O(\log N)$ | Space: $O(1)$

```
// Java Solution Code:
public int search(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (nums[mid] == target) return mid;
        if (nums[l] <= nums[mid]) {
            if (target >= nums[l] && target < nums[mid]) r = mid - 1;
            else l = mid + 1;
        } else {
            if (target > nums[mid] && target <= nums[r]) l = mid + 1;
            else r = mid - 1;
        }
    }
    return -1;
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Binary Search Search Space

Topic 10: Binary Trees & BST — Lowest Common Ancestor (LCA) in Binary Tree

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 8h | **Prerequisite:** Recursion

Senior Interview Question: How to find the Lowest Common Ancestor of two nodes in a Binary Tree?

Real-World Usecase: Navigating hierarchical organization charts and DOM trees.

■ **Coding Challenge:** LeetCode #236 - Lowest Common Ancestor of a Binary Tree | **Complexity:** Time: $O(N)$ | Space: $O(H)$

```
// Java Solution Code:
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

Target Project: Employee Management System | **Resources:** LeetCode Binary Tree Traversals

Topic 11: Binary Search Trees (BST) — BST Validation with Min/Max Bounds

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Binary Trees

Senior Interview Question: How to validate if a Binary Tree is a valid BST in $O(N)$ time?

Real-World Usecase: Validating indexed range search structures.

■ **Coding Challenge:** [LeetCode #98 - Validate Binary Search Tree](#) | **Complexity:** Time: $O(N)$ | Space: $O(H)$

```
// Java Solution Code:
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}
private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) return false;
    return validate(node.left, min, node.val) && validate(node.right, node.val, max);
}
```

Target Project: Employee Management System | **Resources:** [LeetCode Validate BST & LCA](#)

Topic 12: Heaps & Priority Queues — Min-Heap / Max-Heap Internals & Top K Elements Pattern

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Binary Trees & Arrays

Senior Interview Question: How to find Top K Frequent Elements using Min-Heap in $O(N \log K)$?

Real-World Usecase: Building real-time trending leaderboards and task schedulers.

■ **Coding Challenge:** [LeetCode #347 - Top K Frequent Elements](#) | **Complexity:** Time: $O(N \log K)$ | Space: $O(N)$

```
// Java Solution Code:
public int[] topKFrequent(int[] nums, int k) {
    Map map = new HashMap<>();
    for (int n : nums) map.put(n, map.getOrDefault(n, 0) + 1);
    PriorityQueue<Map.Entry> pq =
        new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());
    for (Map.Entry entry : map.entrySet()) {
        pq.add(entry);
        if (pq.size() > k) pq.poll();
    }
    int[] res = new int[k];
    for (int i = 0; i < k; i++) res[i] = pq.poll().getKey();
    return res;
}
```

Target Project: Chat Application | **Resources:** [LeetCode Top K Frequent Elements](#)

Topic 13: Graph Algorithms — Graph Traversals (BFS / DFS & Connected Components)

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Trees & Queues

Senior Interview Question: How to count Number of Islands in a 2D grid using BFS/DFS?

Real-World Usecase: Social network connection degrees and microservice dependency mapping.

■ **Coding Challenge:** LeetCode #200 - Number of Islands | **Complexity:** Time: $O(M * N)$ | Space: $O(M * N)$

```
// Java Solution Code:
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                dfs(grid, r, c);
                count++;
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int r, int c) {
    if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') return;
    grid[r][c] = '0';
    dfs(grid, r+1, c); dfs(grid, r-1, c); dfs(grid, r, c+1); dfs(grid, r, c-1);
}
```

Target Project: Banking System | **Resources:** LeetCode Graph BFS / DFS

Topic 14: Graph Algorithms — Topological Sort (Kahn's BFS Algorithm)

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** Graph Traversals

Senior Interview Question: How to find valid course ordering using Topological Sort (Kahn's In-degree BFS)?

Real-World Usecase: Resolving build dependencies in CI/CD pipelines and task execution ordering.

■ **Coding Challenge:** LeetCode #210 - Course Schedule II | **Complexity:** Time: $O(V + E)$ | Space: $O(V + E)$

```
// Java Solution Code:
public int[] findOrder(int n, int[][] prereqs) {
    List[] adj = new List[n];
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] p : prereqs) {
        adj[p[1]].add(p[0]);
        indegree[p[0]]++;
    }
    Queue q = new LinkedList<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) q.add(i);
    int[] res = new int[n]; int idx = 0;
    while (!q.isEmpty()) {
        int curr = q.poll();
        res[idx++] = curr;
        for (int next : adj[curr]) {
            if (--indegree[next] == 0) q.add(next);
        }
    }
    return idx == n ? res : new int[0];
}
```

Target Project: Banking System | **Resources:** LeetCode Course Schedule I & II

Topic 15: Graph Algorithms — Dijkstra's Shortest Path Algorithm

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Graph BFS & Min-Heap

Senior Interview Question: How to calculate network delay time for all nodes using Dijkstra's Algorithm?

Real-World Usecase: Routing network packets dynamically and logistics delivery route optimization.

■ **Coding Challenge:** LeetCode #743 - Network Delay Time | **Complexity:** Time: $O(E \log V)$ | Space: $O(V + E)$

```
// Java Solution Code:
public int networkDelayTime(int[][] times, int n, int k) {
    Map<Integer, List<Integer>> graph = new HashMap<>();
    for (int[] t : times) {
        graph.computeIfAbsent(t[0], x -> new ArrayList<>()).add(new int[]{t[1], t[2]});
    }
    PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.add(new int[]{k, 0});
    Map<Integer, Integer> dist = new HashMap<>();
    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0], d = curr[1];
        if (dist.containsKey(u)) continue;
        dist.put(u, d);
        if (graph.containsKey(u)) {
            for (int[] edge : graph.get(u)) {
                if (!dist.containsKey(edge[0])) pq.add(new int[]{edge[0], d + edge[1]});
            }
        }
    }
    if (dist.size() != n) return -1;
    int max = 0;
    for (int v : dist.values()) max = Math.max(max, v);
    return max;
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Network Delay Time

Topic 16: Dynamic Programming (DP) — 1D DP & Coin Change Pattern

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Recursion

Senior Interview Question: How to compute minimum number of coins to make target amount using Bottom-up DP?

Real-World Usecase: Currency change breakdown algorithms and resource capacity packing.

■ **Coding Challenge:** LeetCode #322 - Coin Change | **Complexity:** Time: $O(\text{Amount} * N)$ | Space: $O(\text{Amount})$

```
// Java Solution Code:
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int c : coins) {
            if (i >= c) dp[i] = Math.min(dp[i], dp[i - c] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Target Project: Banking System | **Resources:** LeetCode Coin Change 1 & 2

Topic 17: Dynamic Programming (DP) — 2D String DP (Edit Distance)

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 10h | **Prerequisite:** 1D DP

Senior Interview Question: How to compute Edit Distance (Levenshtein Distance) between two strings using 2D DP?

Real-World Usecase: Fuzzy string matching, spell checkers, and diff generation engines.

■ **Coding Challenge:** LeetCode #72 - Edit Distance | **Complexity:** Time: $O(M * N)$ | Space: $O(M * N)$

```
// Java Solution Code:
public int minDistance(String w1, String w2) {
    int m = w1.length(), n = w2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (w1.charAt(i-1) == w2.charAt(j-1)) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + Math.min(dp[i-1][j-1], Math.min(dp[i-1][j], dp[i][j-1]));
        }
    }
    return dp[m][n];
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Edit Distance

Topic 18: Backtracking & Recursion — Combinatorial Search (Subsets Pattern)

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Recursion

Senior Interview Question: How to generate all possible unique subsets of a set of numbers using Backtracking?

Real-World Usecase: Generating permission matrix combinations and feature toggle configurations.

■ **Coding Challenge:** LeetCode #78 - Subsets | **Complexity:** Time: $O(N * 2^N)$ | Space: $O(N)$

```
// Java Solution Code:
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(res, new ArrayList<>(), nums, 0);
    return res;
}

private void backtrack(List<List<Integer>> res, List<Integer> tmp, int[] nums, int start) {
    res.add(new ArrayList<>(tmp));
    for (int i = start; i < nums.length; i++) {
        tmp.add(nums[i]);
        backtrack(res, tmp, nums, i + 1);
        tmp.remove(tmp.size() - 1);
    }
}
```

Target Project: Employee Management System | **Resources:** LeetCode Subsets & Permutations

Topic 19: Advanced Data Structures — Trie (Prefix Tree) Implementation & Search

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Trees & Strings

Senior Interview Question: How to implement `insert`, `search`, and `startsWith` in a Trie with $O(L)$ time complexity?

Real-World Usecase: Building auto-complete search bars and IP routing longest-prefix matching.

■ **Coding Challenge:** LeetCode #208 - Implement Trie (Prefix Tree) | **Complexity:** Time: $O(L)$ | Space: $O(N * L)$

```
// Java Solution Code:
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEnd = false;
}
class Trie {
    private TrieNode root = new TrieNode();
    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) node.children[c - 'a'] = new TrieNode();
            node = node.children[c - 'a'];
        }
        node.isEnd = true;
    }
    public boolean startsWith(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            if (node.children[c - 'a'] == null) return false;
            node = node.children[c - 'a'];
        }
        return true;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** LeetCode Implement Trie

Topic 20: Bit Manipulation — Bitwise Operators & Single Number Pattern

Priority: Medium | **Difficulty:** Intermediate | **Est. Hours:** 5h | **Prerequisite:** Binary Numbers

Senior Interview Question: How to find Single Number in $O(N)$ time and $O(1)$ space using XOR properties?

Real-World Usecase: Storing compact boolean flag sets inside single bitmask integers.

■ **Coding Challenge:** LeetCode #136 - Single Number | **Complexity:** Time: $O(N)$ | Space: $O(1)$

```
// Java Solution Code:
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) result ^= num;
    return result;
}
```

Target Project: Banking System | **Resources:** LeetCode Single Number & Bitmasking

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 21: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:  
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON  
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2  
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 22: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:  
@Entity  
public class Account {  
    @Id @GeneratedValue private Long id;  
    @Version private Long version;  
    private BigDecimal balance;  
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 23: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 24: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 25: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 26: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 27: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 28: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 29: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 30: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 31: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

// Java Solution Code:

```
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 32: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

// Java Solution Code:

```
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 33: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 34: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 35: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 36: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | **Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 37: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 38: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 39: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 40: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 41: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 42: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 43: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 44: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 45: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge:** Spring Boot JWT Auth | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 46: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 47: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 48: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 49: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 50: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 51: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

// Java Solution Code:

```
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 52: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

// Java Solution Code:

```
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 53: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 54: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 55: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 56: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | **Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 57: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 58: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 59: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 60: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 61: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

// Java Solution Code:

```
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 62: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

// Java Solution Code:

```
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 63: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 64: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 65: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 66: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | **Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 67: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 68: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 69: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 70: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 71: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 72: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 73: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 74: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 75: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 76: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 77: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 78: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 79: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 80: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 81: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: $O(N^2)$ | Space: $O(N)$**

// Java Solution Code:

```
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 82: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge | Complexity: Time: $O(1)$ | Space: $O(1)$**

// Java Solution Code:

```
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 83: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 84: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 85: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 86: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | **Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 87: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 88: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 89: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 90: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 91: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

// Java Solution Code:

```
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 92: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

// Java Solution Code:

```
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 93: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 94: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 95: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 96: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 97: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 98: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 99: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 100: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 101: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: $O(N^2)$ | Space: $O(N)$**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 102: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 103: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 104: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 105: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 106: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 107: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 108: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 109: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 110: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 111: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 112: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 113: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 114: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 115: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 116: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 117: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 118: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 119: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 120: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 121: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 122: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 123: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 124: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 125: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 126: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 127: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 128: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 129: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 130: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 131: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 132: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 133: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 134: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 135: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 136: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 137: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 138: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 139: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 140: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 141: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 142: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 143: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 144: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 145: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 146: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 147: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 148: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 149: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 150: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 151: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:  
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON  
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2  
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 152: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:  
@Entity  
public class Account {  
    @Id @GeneratedValue private Long id;  
    @Version private Long version;  
    private BigDecimal balance;  
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 153: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 154: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 155: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 156: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 157: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 158: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 159: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 160: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 161: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:  
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON  
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2  
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 162: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:  
@Entity  
public class Account {  
    @Id @GeneratedValue private Long id;  
    @Version private Long version;  
    private BigDecimal balance;  
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 163: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 164: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 165: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 166: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 167: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 168: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 169: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 170: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 171: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 172: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 173: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 174: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 175: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 176: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 177: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 178: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 179: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 180: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 181: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 182: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 183: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 184: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 185: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 186: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 187: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 188: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 189: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 190: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 191: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 192: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 193: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 194: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 195: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 196: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 197: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 198: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 199: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 200: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 201: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 202: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 203: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 204: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 205: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 206: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 207: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 208: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 209: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 210: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 211: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries** | Complexity: Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 212: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 213: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 214: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 215: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 216: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 217: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 218: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 219: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 220: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 221: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 222: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 223: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 224: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 225: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 226: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 227: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 228: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 229: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 230: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 231: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 232: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 233: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 234: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 235: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 236: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 237: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 238: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 239: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 240: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 241: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 242: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 243: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 244: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 245: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 246: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 247: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 248: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 249: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 250: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 251: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 252: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 253: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 254: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 255: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 256: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | Difficulty: Advanced | Est. Hours: 7h | Prerequisite: SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge: LeetCode #185 - Department Top Three Salaries | Complexity: Time: O(N^2) | Space: O(N)**

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 257: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge: JPA Lock Challenge** | Complexity: Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 258: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals** | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 259: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | **Difficulty:** Intermediate | **Est. Hours:** 6h | **Prerequisite:** Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 260: Spring Security — JWT Authentication Filter Setup

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth** | **Complexity:** Time: O(1) | Space: O(1)

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup

Phase 2: Database Engineering, SQL Deep Dive & NoSQL

Topic 261: Advanced SQL — SQL Window Functions ROW_NUMBER vs DENSE_RANK

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 7h | **Prerequisite:** SQL Joins

Senior Interview Question: Write a query to find Top 3 highest paid employees per department.

Real-World Usecase: Production implementation pattern for SQL Window Functions ROW_NUMBER vs DENSE_RANK.

■ **Coding Challenge:** LeetCode #185 - Department Top Three Salaries | **Complexity:** Time: $O(N^2)$ | Space: $O(N)$

```
// Java Solution Code:
SELECT d.Name AS Department, e.Name AS Employee, e.Salary FROM Employee e JOIN Department d ON
e.DepartmentId = d.Id WHERE 3 > (SELECT COUNT(DISTINCT e2.Salary) FROM Employee e2 WHERE e2
.Salary > e.Salary AND e2.DepartmentId = e.DepartmentId);
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for SQL Window Functions ROW_NUMBER vs DENSE_RANK

Topic 262: Database Concurrency — Pessimistic vs Optimistic Locking

Priority: High | **Difficulty:** Advanced | **Est. Hours:** 8h | **Prerequisite:** ACID Transactions

Senior Interview Question: How to implement Optimistic Locking with JPA @Version annotation?

Real-World Usecase: Production implementation pattern for Pessimistic vs Optimistic Locking.

■ **Coding Challenge:** JPA Lock Challenge | **Complexity:** Time: $O(1)$ | Space: $O(1)$

```
// Java Solution Code:
@Entity
public class Account {
    @Id @GeneratedValue private Long id;
    @Version private Long version;
    private BigDecimal balance;
}
```

Target Project: Banking System | **Resources:** Official Guide & Solution Blueprint for Pessimistic vs Optimistic Locking

Phase 3: Java Core & Advanced Engineering

Topic 263: Java Collections — HashMap Internals & Treeify Threshold

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: hashCode Equals

Senior Interview Question: Why Java 8 converts HashMap buckets to Red-Black Trees at length 8?

Real-World Usecase: Production implementation pattern for HashMap Internals & Treeify Threshold.

■ **Coding Challenge: Java Core HashMap Internals | Complexity: Time: $O(1)$ avg, $O(\log N)$ worst | Space: $O(N)$**

```
// Java Solution Code:
public V put(K key, V value) {
    return putVal(hash(key), key, value, false, true);
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for HashMap Internals & Treeify Threshold

Topic 264: Java Multithreading — Thread Safe Singleton Double-Checked Locking

Priority: High | Difficulty: Intermediate | Est. Hours: 6h | Prerequisite: Multithreading

Senior Interview Question: Write a thread-safe Singleton using Double-Checked Locking in Java.

Real-World Usecase: Production implementation pattern for Thread Safe Singleton Double-Checked Locking.

■ **Coding Challenge: Design Pattern Singleton | Complexity: Time: $O(1)$ | Space: $O(1)$**

```
// Java Solution Code:
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for Thread Safe Singleton Double-Checked Locking

Phase 4: Spring Boot & Enterprise Backend Architecture

Topic 265: Spring Security — JWT Authentication Filter Setup

Priority: High | Difficulty: Advanced | Est. Hours: 8h | Prerequisite: Spring Security

Senior Interview Question: How to intercept requests and validate JWT Bearer token in Spring Security filter?

Real-World Usecase: Production implementation pattern for JWT Authentication Filter Setup.

■ **Coding Challenge: Spring Boot JWT Auth | Complexity: Time: O(1) | Space: O(1)**

```
// Java Solution Code:
public class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            // validate token & set SecurityContext
        }
        chain.doFilter(req, res);
    }
}
```

Target Project: E-Commerce Platform | **Resources:** Official Guide & Solution Blueprint for JWT Authentication Filter Setup